Task 12 — Generalization, Robustness, and Anomaly Detection

1. Introduction and Problem Statement

This report investigates a question that standard machine learning workflows often skip: is a model's reported performance actually trustworthy, or is it an artifact of a convenient train/test split? Using a RandomForestClassifier — the same model family used in Task 11 — this study evaluates the same trained model under multiple evaluation conditions of increasing realism, checks whether its performance is stable across repeated runs, explores what structure the data reveals without any labels at all, and deliberately stresses the model against a plausible real-world failure mode.

The aim throughout is not to maximize a single accuracy number, but to determine which performance claims about the model are actually defensible once convenient assumptions are removed.

2. Dataset Discovery and Justification

The UCI Occupancy Detection Dataset (Candanedo & Feldheim, 2016) was selected for this task. It records five continuous ambient sensor readings — Temperature, Humidity, Light, CO2, and HumidityRatio — in an office room, with a binary Occupancy label (0 = empty, 1 = occupied) ground-truthed from time-stamped photographs.

- Rows: a single row is one timestamped sensor reading (approximately one per minute).
- Files: datatraining.txt (8,143 rows, Feb 4–10, 2015), datatest.txt (2,665 rows, Feb 2–4, 2015, collected with the office door open), and datatest2.txt (9,752 rows, Feb 11–18, 2015).
- License: publicly available via the UCI Machine Learning Repository for research use, with citation to Candanedo, L. and Feldheim, V. (2016).

This dataset was chosen specifically because its three files are not an arbitrary random split — they were collected on genuinely different days under genuinely different conditions. datatest.txt precedes the training period and was collected with the door open (altering ventilation and CO2 behaviour); datatest2.txt follows the training period chronologically. This gives two independent, real forms of distribution shift to test against — temporal (later, unseen days) and environmental (a different physical condition) — without needing to synthesize either artificially.

Class balance differs meaningfully across the three files: training data is 78.8% empty / 21.2% occupied, test2 is very similar (79.0% / 21.0%), but test1 (door-open) is markedly more balanced at 63.5% / 36.5% — itself informative, since it means the door-open condition also changed how often the room was actually occupied during measurement, not only how the sensors read.

3. Experimental Setup

A RandomForestClassifier (n_estimators=200) was trained on the five raw sensor features to predict Occupancy. Two evaluation strategies were used throughout:

- Reference (naive) split: all rows from all three files pooled together, then a random 80/20 stratified train/test split.
- Realistic splits: the model trained only on datatraining.txt, then evaluated separately on datatest2.txt (chronological / future-day generalization) and datatest.txt (condition-shift / door-open generalization) — both fully held out, never seen during training.

Accuracy and F1 were both tracked throughout, since Occupancy is imbalanced (~79/21); F1 is more sensitive to degradation on the minority (occupied) class than accuracy alone.

4. Reference vs. Realistic Evaluation

The same model architecture was evaluated under all three conditions described above.

Evaluation	Accuracy	F1
Random split (naive)	0.9908	0.9801
Realistic — future days (test2)	0.9728	0.9380
Realistic — condition shift, door-open (test1)	0.9493	0.9294

The naive random split reports the highest scores (98.0% F1) because rows from the same day — often minutes apart — land in both train and test, letting the model implicitly see conditions from the same period it is tested on. The realistic future-day split (test2) shows a meaningful drop (93.8% F1), and the condition-shift split (test1, door-open) drops further still (92.9% F1) despite covering fewer days than test2. This indicates that a genuine change in environmental condition is a harder generalization problem for this model than the mere passage of time, and that the random split's headline accuracy substantially overstates real-world performance.

5. Stability Analysis (Repeated Runs)

To check whether the Section 4 results reflect stable model behaviour or a lucky/unlucky single run, both the random split and the realistic (test2) split were repeated across 5 different random seeds, varying only the model's internal randomness for the realistic split (since its train/test dates are fixed by design) and both the split composition and model randomness for the random split.

Split	Mean F1 (5 seeds)	Std F1
Random split	0.9845	0.0032
Realistic split (test2)	0.9388	0.0098

The realistic split is not only lower-scoring on average, it is roughly three times more variable (std 0.0098 vs. 0.0032) than the random split, even though only model-internal randomness changed — the evaluation data itself was held constant. This suggests the harder, more realistic evaluation condition also makes the model's behaviour less predictable from run to run. Conversely, the random split's tight variance should not be read purely as good news: consistently high scores under an evaluation condition that leaks information across train and test reflects consistent overoptimism, not consistent real-world reliability.

6. Unsupervised Structure: Clustering and Anomaly Detection

6.1 K-Means Clustering

K-Means (k=2) was fit on standardized features with no access to the Occupancy label, to test whether the data naturally separates into groups resembling occupied/empty on its own. Occupancy was compared against the resulting clusters only afterward.

- Cluster 0: 98.2% empty, 1.8% occupied — a near-pure ‘empty room’ cluster.
- Cluster 1: 69.4% occupied, 30.6% empty — skews occupied but with substantial overlap.

Natural clustering partially aligns with occupancy but is not a clean substitute for a supervised classifier — a meaningful fraction of empty-room readings resemble occupied-room readings closely enough to fall into the same cluster, most plausibly rooms shortly after being vacated, when CO2 and temperature have not yet returned to baseline.

6.2 IsolationForest Anomaly Detection

IsolationForest was fit on the same standardized features, again without using Occupancy. It isolates points via random recursive splitting; points that require fewer splits to isolate (shorter average path length across the forest) are scored as more anomalous, since outliers sit in sparser regions of the feature space than the dense clusters of typical rows.

At contamination=0.05 (the initial setting), 1,027 of 20,560 rows were flagged as anomalies. Comparing against the true labels afterward: 56.2% of flagged anomalies were occupied rows, versus a 21.2% occupied rate in the full dataset — anomalous sensor readings are considerably more likely to occur when the room is occupied, consistent with occupied conditions producing more varied, less repetitive sensor patterns than an empty room's steady baseline.

6.3 Contamination Sensitivity

Contamination	# Flagged	Occupied-rate among flagged	vs. baseline (21.2%)
0.02 (strictest)	412	75.2%	+54.0 pts
0.05	1,027	56.2%	+35.0 pts
0.10 (loosest)	2,056	61.0%	+39.8 pts

The occupied-skew among flagged anomalies holds at every threshold tested, confirming the finding is not an artifact of one particular cutoff. The relationship is strongest, however, at the strictest threshold (0.02): the very oddest readings are 75.2% occupied, while loosening the threshold to include more borderline points brings this down to the 56–61% range. This should not be read as strictly monotonic — 0.10 shows a slightly higher rate than 0.05 — so the more precise claim is that the most confidently anomalous points carry the strongest occupied-skew, while the relationship weakens and becomes noisier among borderline points. It is also important to state plainly that IsolationForest detects statistical rarity, not occupancy directly; the correlation between the two is a property of this dataset's behaviour, not evidence that anomaly detection can substitute for the trained classifier.

7. Robustness Stress Test

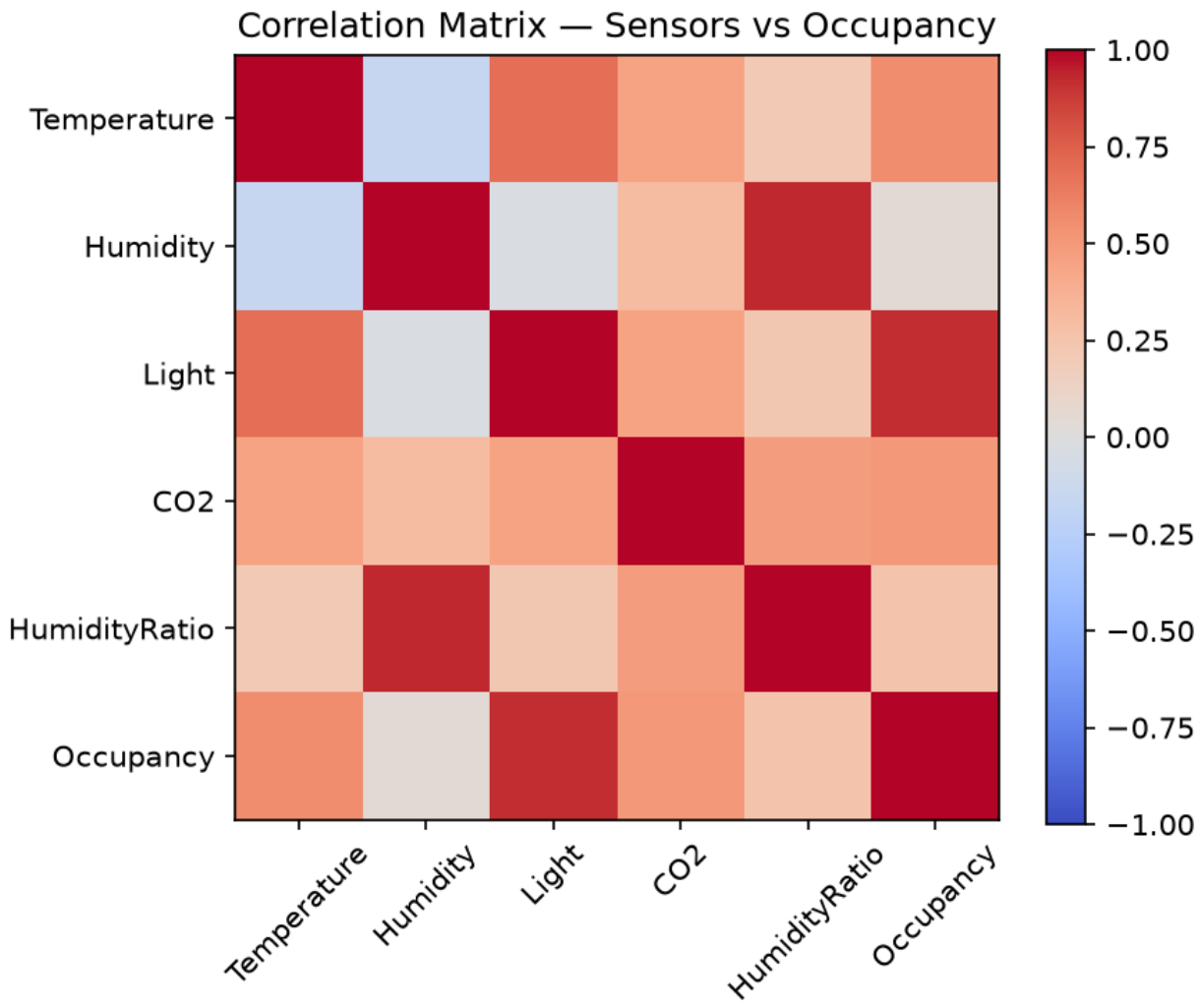
A plausible real-world failure was simulated: CO2 sensor malfunction, represented by replacing the CO2 reading in the test2 evaluation set with a constant fallback value (the training-set mean), while leaving the trained model and all other features untouched. Only the test data was corrupted; the model was not retrained.

Condition	Accuracy	F1
Clean test2	0.9728	0.9380
CO2-failure (flattened to train mean)	0.9884	0.9726

Contrary to the intuitive expectation that removing a feature's real signal should hurt performance, accuracy improved from 97.3% to 98.8% and F1 improved from 93.8% to 97.3% once CO2 was flattened. This is explained by feature drift, examined next.

8. Feature Importance and Drift Analysis

Feature	Importance
Light	0.6204
CO2	0.2329
Temperature	0.0887
HumidityRatio	0.0317
Humidity	0.0263



CO2 was the model's second most heavily used feature (23.3% importance, behind only Light at 62.0%). Checking its distribution across periods explains the robustness result: mean CO2 was 606.5 ppm in the training period versus 753.2 ppm in test2, a roughly 24% rise in baseline level over the 11-day gap between periods, with comparable spread (std ~300–314 ppm in both). The model had learned occupancy thresholds calibrated to training-period CO2 levels; by test2, CO2 had drifted upward even during genuinely empty periods, causing the model to misread elevated-but-empty rooms as occupied. Neutralizing CO2 removed this misleading, drifted signal and forced the model to rely on the other four, more stable features, which is why performance improved rather than declined.

This is a materially different and more instructive finding than a simple ‘corrupting a feature hurts performance’ result: a feature that is both heavily relied upon and prone to drift can become a liability at deployment time even without any sensor malfunction, purely because its distribution shifted between training and deployment periods.

9. Limitations and Risks

- The naive random split overstates real-world performance by an estimated 4–5 F1 points due to temporal leakage between train and test rows drawn from the same or nearby timestamps.
- Performance degrades further, and becomes less stable, under a genuine environmental condition shift (door-open) than under a simple chronological shift, indicating the model is more sensitive to changes in the underlying data-generating conditions than to the mere passage of time.

- CO2, one of the two most important features, is not stable over time in this environment; any deployment of this model should monitor for CO2 baseline drift rather than assuming the training-time relationship between CO2 and occupancy holds indefinitely.
- Unsupervised methods (K-Means, IsolationForest) surface structure that correlates with occupancy but should not be treated as label-free substitutes for the supervised classifier — both leave substantial overlap or ambiguity that only the labeled model resolves cleanly.
- Only one robustness scenario (CO2 sensor flatlining) was tested; other plausible failures — e.g. Light sensor failure, given it is the dominant feature at 62% importance — were not evaluated and would be a natural extension.

10. Conclusion — Which Performance Claims Are Defensible

The headline 99.1% accuracy from a naive random split is not a defensible claim about real-world deployment performance; it is inflated by data leakage between nearby timestamps. The more defensible claims, established through realistic held-out evaluation, are: the model achieves approximately 93–97% F1 on genuinely unseen future days, degrading further to approximately 93% F1 under a materially different environmental condition it has not encountered before. Its performance under realistic conditions is also meaningfully less stable across runs than the optimistic random-split number suggests.

Separately, the model's second-most-relied-upon feature (CO2) is demonstrably unstable over the timescales examined, meaning even the realistic-split numbers above should be treated as representative of the specific 11-day window tested rather than a permanent guarantee — continued monitoring of feature drift would be necessary for any real deployment. Unsupervised analysis independently corroborates that unusual sensor readings skew toward occupied conditions, offering a modest, label-free sanity check on the supervised model's behaviour, but not a replacement for it. Taken together, this study's methodology — comparing evaluation strategies, repeating runs, probing without labels, and deliberately stressing a known dependency — produces a substantially more honest and defensible characterization of model performance than a single train/test split ever could.